

## Quiz 3 – Applied OS: Cyber-Defense & Agentic AI

L2: Student Name: \_\_\_\_\_

| Q1  |     | Total |
|-----|-----|-------|
| (5) | (5) | (10)  |
|     |     |       |

Q1 Program 5.1:

(5)

```
a) #include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p = malloc(sizeof(int));

    free(p);

    *p = 5;

    printf("[%d] - Done.\n", *p);
    return 0;
}
```

Program 5.1 prints the following:

```
[5] - Done.
```

Program 5.1 should have given an error, but it doesn't; rather, it seems to work fine. Why?

- “free(p)” releases the heap block back to the allocator, but it does **not** erase the memory, does **not** set “p” to NULL, and does **not** invalidate the pointer's address.
- Meaning, “p” still points to the same location, which is usually still part of the process's heap and remains writable. So “\*p = 5” writes into freed memory, and printf reads it back as 5.
- This is **undefined behavior** — it only appears to work because the memory hasn't been reused or unmapped yet. The program could just as easily crash or print garbage on another system, compiler, or run. Compiling with AddressSanitizer would report a heap-use-after-free error.

b) Program 5.2:

(5)

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *p = malloc(sizeof(int));
    free(p);

    *p = 5;

    printf("[%d] - Done.\n", *p);
    free(p);

    return 0;
}
```

Here we are doing a double free(). Is this a compile-time or a run-time error?

- The double free(p) is a **run-time error** (specifically, undefined behavior that typically causes a crash at runtime).
- **Note** that the compiler only checks syntax and basic semantics. It does not track the dynamic state of memory. As far as the compiler is concerned, p is still a valid `int * pointer`, so calling `free(p)` twice looks syntactically correct.
- At runtime, the memory allocator (e.g., glibc's malloc/free) keeps metadata about which blocks are currently allocated and which are free. When free(p) is called the first time, the block is marked as free. When free(p) is called the second time, the allocator detects that the block is already free (a double free) and typically calls abort(), terminating the program. Other times, it may not give an error; hence the term “undefined behavior.”